

Agency OS — Handoff for David

Beta feedback items that need infrastructure / Google Cloud work

Prepared 24 Aug 2026, updated 25 Aug 2026 · from beta tester feedback (Agency OS Issues and suggestions; Vlad's Notes)

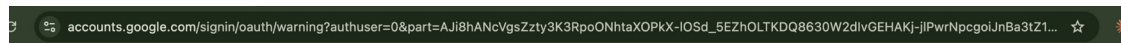
The app-side code for these has already been done and is on the **staging** branch. This doc covers only the six items that need **you** — the parts we can't do from the codebase (Google Cloud Console settings, verification, and integrations that need external accounts/keys). Each item has the customer's own words, a screenshot where there is one, the cause, and exactly what's needed.

1 · “This app is blocked” when connecting Search Console

OWNER: DAVID — Google Cloud Console

What the customer said

“Trying to add Search Console, but got this error.”



This app is blocked

This app tried to access sensitive info in your Google Account. To keep your account safe, Google blocked this access.

Google's “This app is blocked” screen the tester hit (*accounts.google.com* OAuth warning).

What's going on

Two causes stacked up. **(a) Code — already fixed:** the Integrations page asked Google for the sensitive Analytics scope bundled together with Search Console, so Google blocked the whole consent screen on our unverified app. That button is now split so Search Console (a non-sensitive scope) can be granted on its own. **(b) Config — yours:** our OAuth app's publishing status controls who can connect at all.

What you need to do (one-time, applies to all users)

In **Google Cloud Console** → **APIs & Services** → **OAuth consent screen** for the project that owns our OAuth client:

- Fast path for the 5 beta users: add each of their Google emails under “Test users”. They can connect immediately. Downside: in “Testing” mode Google expires every connection after 7 days, so they'd have to reconnect weekly.
- Recommended for launch: click “Publish app” to move the consent screen to Production. For the scopes Search Console and Drive use (non-sensitive) this needs NO Google review, and it removes the 7-day expiry.

- Only the Analytics scope (analytics.readonly) is “sensitive” and would need Google’s formal OAuth verification. We do NOT need it today — GA and Search Console data both come through Windsor.ai — so this is optional and can wait.

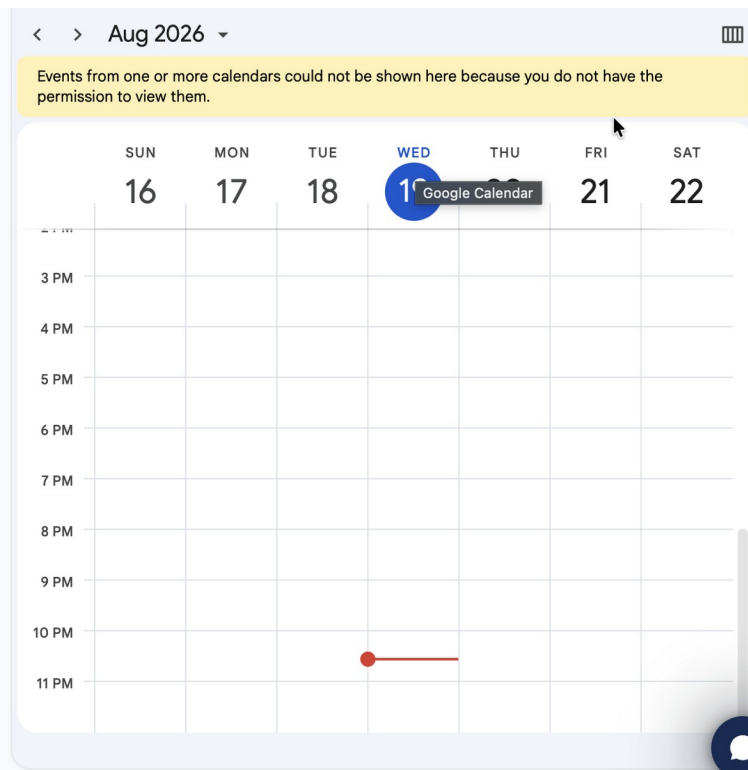
Repo context (already done, for reference): core/src/security/google-oauth.ts (scope classification) and web/src/app/(hosted)/integrations/page.tsx (split connect buttons).

2 · Calendar not showing (“you do not have permission”)

OWNER: DAVID — build native Calendar integration

What the customer said

“Calendar not showing; I’m not sure what to do here to resolve it.”



The calendar panel showing Google’s “Events... could not be shown here because you do not have the permission to view them.”

What’s going on

The calendar panel today is a **public Google Calendar embed (an iframe keyed by the user’s email)**, not an API integration. That embed only renders a private calendar when the same browser is signed in to that exact Google account and allows third-party cookies for calendar.google.com. Modern browsers block third-party cookies by default, so the iframe is treated as anonymous and Google returns the permission error. It can’t be fixed with a setting — the approach itself is the limit. (An in-app help note explaining the interim workaround is already shipped.)

What needs to be built

- Replace the iframe with the Google Calendar API: add a calendar.readonly scope to our existing Google OAuth, fetch the signed-in user's events, and render them in the app's own calendar UI.
- Note: calendar.readonly is a sensitive scope, so it lands on the same OAuth-verification consideration as item 1 — worth batching with that decision.

Rough size: ~1–2 days. Repo pointers: web/src/components/hosted/calendar-embed.tsx (current iframe) and core/src/security/google-oauth.ts (add the scope).

3 · Bring-your-own local rank-tracking (gtrack.wiremo.co)

OWNER: DAVID — scope a new provider integration

What the customer said

"Is there a way to add our own APIs? Or can we upload our own local map grids for reporting? I use <https://gtrack.wiremo.co/> instead of local falcon, just because its cheaper."

What's being asked

The agency wants to use their own local rank-grid tool (gtrack.wiremo.co) in place of Local Falcon, which is what Agency OS integrates with today for local map-grid reporting.

What needs to be scoped/built

- Add gtrack as an alternative local-grid provider alongside Local Falcon: a provider adapter that maps gtrack's data into our report shape, per-agency API-key storage (we already have the encrypted credential vault used for Windsor, so that part is cheap), and UI to pick the provider / upload grids.
- Blockers before we can build: we need to know what gtrack actually exposes (a documented API? CSV/grid exports only?), and a gtrack account / API key to develop and test against.
- Recommendation: scope gtrack specifically as a bounded fast-follow, rather than a generic "add any API" system, which is a much larger surface.

Repo pointer: core/src/analysis/providers/localfalcon.ts is the existing local-grid provider to model the adapter on.

4 · Drive Picker — "The API developer key is invalid"

OWNER: DAVID — Google Cloud Console

What the customer said

"When I clicked Choose from Drive (Link Workbook) error came up (The API developer key is invalid)."

ytics — expect this to fail until that is in place.

There was an error!

The API developer key is invalid.

The Drive Picker error the tester hit when choosing a file from Google Drive (“There was an error! The API developer key is invalid.”).

What’s going on

The “Choose from Drive” flow uses Google’s Drive Picker, which authenticates with a browser API key we pass as `GOOGLE_PICKER_API_KEY`. The key is set but Google is rejecting it — which almost always means the Google Picker API isn’t enabled on the project that owns the key, or the key has HTTP-referrer / API restrictions that don’t include our app’s domain. It’s a key-configuration issue, not app code.

What you need to do (Google Cloud Console, same project as our OAuth client):

- APIs & Services — Library — enable the Google Picker API.
- APIs & Services — Credentials — open the key used for `GOOGLE_PICKER_API_KEY`. Under Application restrictions set None, or if you keep HTTP referrers, add our production domain (and any preview/localhost origins). Under API restrictions, make sure Google Picker API is allowed.

Once the key is valid the existing “Choose from Drive” button works with no redeploy on our side.

5 · Auto-created client folder lands at the top of Drive

OWNER: DAVID — decision, then me

What the customer said

“Auto creating a client folder puts it at the top level of Google Drive— we should be able to select a clients or projects folder for Agency OS to operate in.”

What’s going on

When the app creates a client’s Drive folder it lands at the root of the connected Drive. That’s a direct consequence of the OAuth scope we use: `drive.file` only lets the app see and manage folders it created itself — it can’t “see” a pre-existing “Clients” folder to nest new folders inside. To place new client folders under an existing parent, the user has to pick that parent through the Drive Picker (which grants the app access to just that one folder). So this is blocked by item 4 — the picker has to work first.

What’s needed:

- (David) Fix the picker key in item 4. That unblocks this.
- (Me) Then I add a “parent folder” setting so new client folders are created inside the folder you pick, instead of at the Drive root. No further Google config needed after item 4.

6 · Task owner shows a generic icon, not the person’s photo

OWNER: DAVID — confirm Clerk photos, then me

What the customer said

“When you select a task owner under Client / Tasks, you can’t tell who the person is— it’s not using the profile picture, just an icon.”

STATUS	TASK	DELIVERABLE	OWNER	R
Scheduled	Set the client website	ONBOARDING		
Scheduled	Send invoice to client	ONBOARDING		
Scheduled	Send onboarding email	ONBOARDING		
Scheduled	Schedule kickoff call	ONBOARDING		
Scheduled	Get access to client platforms	ONBOARDING		

The Tasks table Owner column showing generic avatars instead of each teammate’s profile photo.

What’s going on

Owner avatars come from Clerk (each member’s profile image). If a member has no photo set on their Clerk account, there’s nothing for the app to render, so it falls back to a plain icon.

What’s needed:

- (David / config) Confirm your beta team members have profile images on their Clerk accounts (or that Clerk is pulling them from Google SSO). No photo in Clerk = nothing to show.
- (Me / code) I’ll improve the fallback so that when there’s no photo it shows the person’s colored initials instead of a generic icon.

At a glance

- Item 1 — Search Console block: publish the OAuth app to Production (or add test users). No build. ~15 min in Google Cloud Console.
- Item 2 — Calendar: build a native Calendar API view (adds calendar.readonly scope). ~1–2 days.
- Item 3 — gtrack local grids: scope a new provider; needs gtrack API details + a key first.
- Item 4 — Drive Picker “key invalid”: enable the Google Picker API + fix the key’s restrictions in Cloud Console. No build, ~10 min.

- Item 5 — Client folder at Drive root: after the picker works, I add a “parent folder” setting. Blocked by item 4.
- Item 6 — Owner shows an icon: confirm the team have Clerk photos; I’ll add an initials fallback.

Everything else from the beta feedback is already fixed and on staging.